

进程间通信

1、进程间通信 (IPC, InterProcess Communication)

每个进程都各自有不用的用户地址空间，任何一个进程的全局变量在另一个进程中都看不到，所以进程之间要交换数据必须通过内核，在内核中开辟一块缓冲区，进程1 把数据从用户空间拷到内核缓冲区，进程2 再从内核缓冲区把数据读走，内核提供的这种机制称为进程间通信。

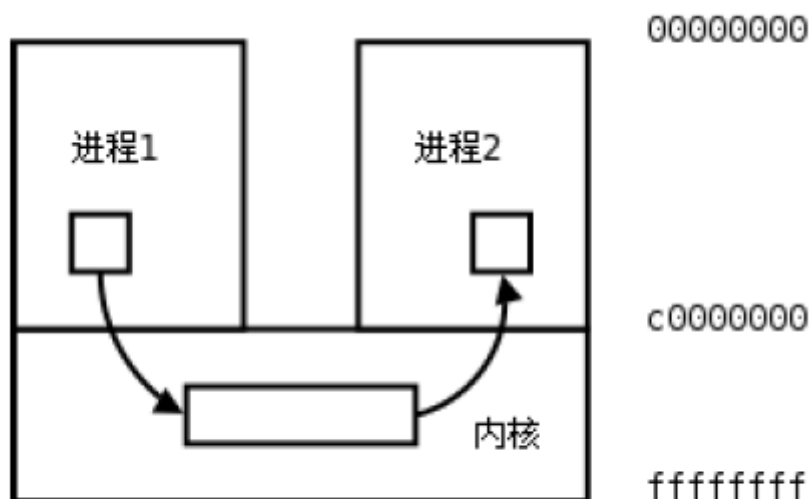
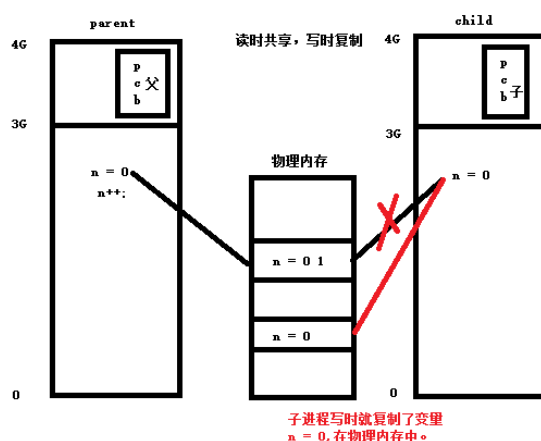


图 4.1: 进程间通信

2、进程的读时共享，写时复制

创建进程或者线程都会生成PCB数据，所有在Linux创建进程和线程的开销都是差不多的。

父进程与子进程，先执行，不一定。取决于CPU的调度算法。



3、PIPE管道

1、管道是一种最基本的IPC通信机制（属于单工通信），由pipe函数创建：

```

1 #include <unistd.h>
2
3
4 int pipe(int filedes[2]); //filedes是传出的数组
5 //返回值：成功返回 0，失败返回 -1，并设置errno.

```

管道作用于有血缘关系的进程之间，通过fork来传递。管道在内核中只有一份。

fork之后，子进程会复制父进程的原来打开的文件描述符，也就是打开的管道文件描述符，管道的读端和写端的文件描述符引用计数 变成了 2。

0读 1写

```

1 #include <stdlib.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5
6 void sys_err(char *str)
7 {
8     perror(str);
9     exit(1);
10 }
11
12
13 int main(void)
14 {
15     int pfd[2], len;
16     char buf[1024];
17
18
19     if(pipe(pfd) < 0)
20         sys_err("pipe");
21     //往管道的写端写入数据
22     write(pfd[1], "hello", 5);
23     //从管道的读端读出数据
24     len = read(pfd[0], buf, sizeof(buf));
25     //写到标准输出上
26     write(STDOUT_FILENO, buf, len);

```

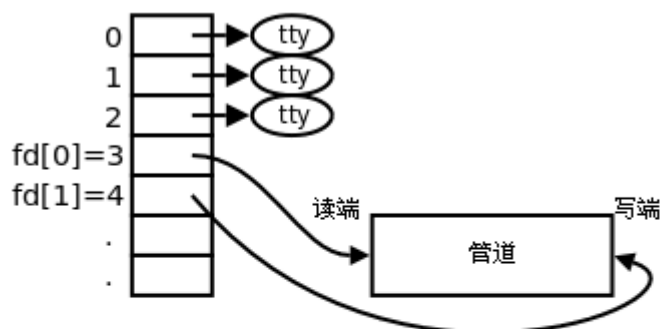
```

26     write(STDOUT_FILENO, buf, len);
27
28
29     return 0;
30 }

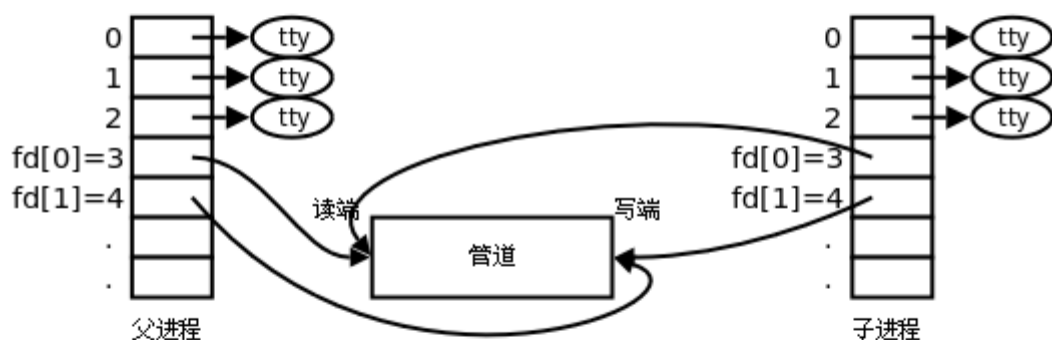
```

2、管道的创建

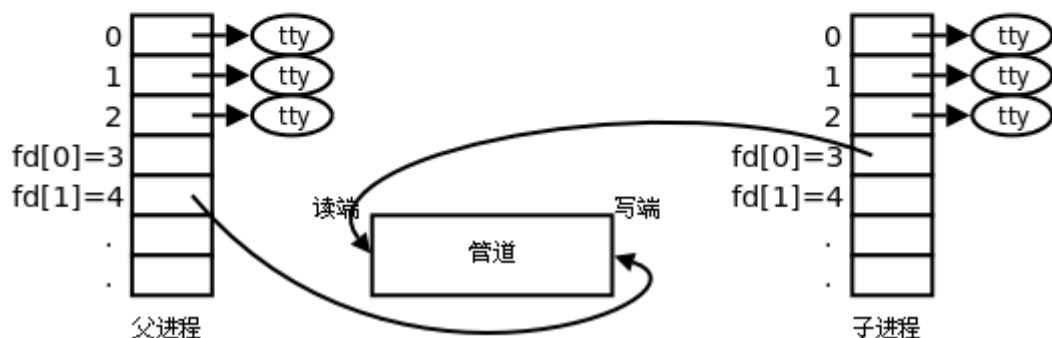
1. 父进程创建管道



2. 父进程fork出子进程



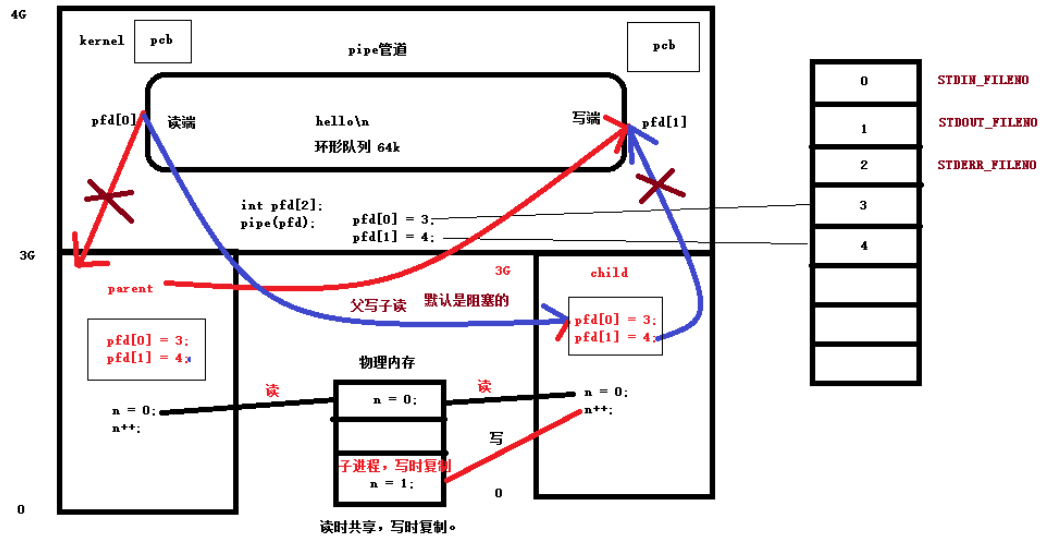
3. 父进程关闭fd[0]，子进程关闭fd[1]



1、父进程调用pipe开辟管道，得到两个文件描述符指向管道的两端。

2、父进程调用fork创建子进程，那么子进程也有两个文件描述符指向同一管道。

3、父进程关闭管道读端，子进程关闭管道写端。父进程可以往管道里写，子进程可以从管道里读，管道是用环形队列实现的，数据从写端流入从读端流出。



```

1  /*管道通信*/
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <stdio.h>
6
7
8  void sys_err(char *str)
9  {
10     perror(str);
11     exit(1);
12 }
13
14
15 int main(void)
16 {
17     int pfd[2], len, i = 0;
18     char buf[1024];
19     pid_t pid;
20
21
22     if(pipe(pfd) < 0)
23         sys_err("pipe");

```

```

24
25
26     pid = fork();
27     if(pid > 0)
28     {
29         //父进程关闭读端0
30         close(pfd[0]);
31         while(1)
32         {
33             sprintf(buf,"yzmin %d\n",i++);
34             write(pfd[1],buf,strlen(buf));
35             sleep(1);
36         }
37     }
38     else if(pid == 0)
39     {
40         //子进程关闭写端1
41         close(pfd[1]);
42         while(1)
43         {
44             len = read(pfd[0],buf,sizeof(buf));
45             write(STDOUT_FILENO,buf,len);
46         }
47     }
48     else
49     {
50         sys_err("fork");
51     }
52
53
54     return 0;
55 }

```

```

1  /*双管道通信*/
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5  #include <sys/stat.h>
6  #include <sys/types.h>
7  #include <string.h>
8  #include <fcntl.h>

```

```

9
10
11 void sys_err(char *str)
12 {
13     perror(str);
14     exit(-1);
15 }
16
17
18 int main(void)
19 {
20     int pfd1[2];
21     int pfd2[2];
22     int pi1 = pipe(pfd1);
23     int pi2 = pipe(pfd2);
24     char buf[1024];
25     int len,i = 0;
26     if((pi1<0)|| (pi2<0))
27     {
28         sys_err("pipe fail");
29     }
30     pid_t pid;
31     pid = fork();
32     if(pid > 0)
33     {
34         while(1)
35         {
36             //父读子写
37             len = read(pfd2[0],buf,sizeof(buf));
38             write(STDOUT_FILENO,buf,len);
39             sleep(2);
40             //父写子读
41             sprintf(buf,"parent send yzmin %d\n",i++);
42             write(pfd1[1],buf,strlen(buf));
43             sleep(2);
44         }
45     }
46     else if(pid == 0)
47     {
48         int flags;
49         while(1)
50         {
51             //设置非阻塞，防止互掐
52             flags = fcntl(pfd1[0], F_GETFL);

```

```

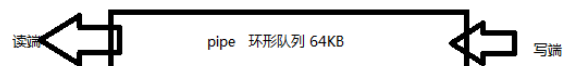
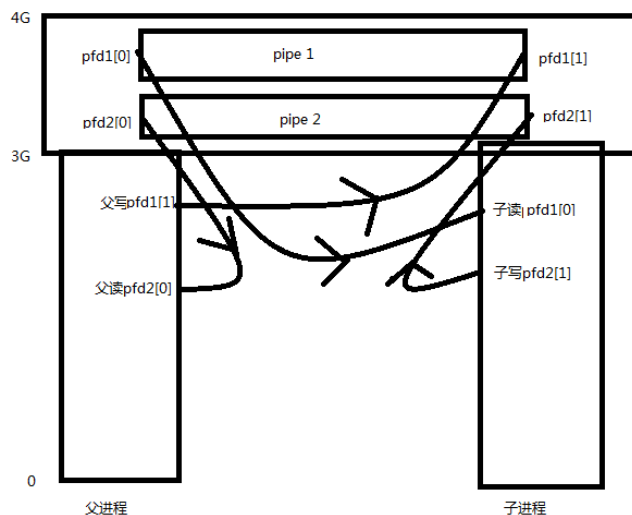
52         flags = fcntl(pfd1[0], F_GETFL);
53         flags |= O_NONBLOCK;
54         fcntl(pfd1[0], F_SETFL, flags);
55         //子读父写
56         len = read(pfd1[0], buf, sizeof(buf));
57         write(STDOUT_FILENO, buf, len);
58         sleep(3);
59         //子写父读
60         sprintf(buf, "child send hello %d\n", i++);
61         write(pfd2[1], buf, strlen(buf));
62         sleep(3);
63     }
64 }
65 else
66 {
67     sys_err("fork");
68 }
69 return 0;
70 }

```

4、管道（pipe）注意事项

管道是单工通信，只能一方写，一方读。

如果两个进程同时进行读写操作，比如父进程即读数据又写数据，那么在子进程读的时候就会出现数据不能读全，因为父进程读走了一些数据导致的。在管道编程中，就需要保持单工通信，保证通信安全。



- 1、写端关闭，读端读完管道剩余的数据，再次读，相当于读到文件末尾，返回0。
- 2、读端关闭，写端写管道，写进程会收到一个SIGPIPE信号，此信号导致写进程终止。
- 3、读端没有读数据，写端写管道，写满管道后，再次写，写阻塞。
- 4、写端没有写管道，读端去读管道，读端读完管道里数据后，再次读，阻塞。

非阻塞管道，fcntl函数设置 O_NONBLOCK标志

fpathconf(int fd,int name) 测试管道缓冲区大小, _PC_PIPE_BUF

1、nonblock_pipe.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7
8
9 void sys_err(const char *str)
10 {
11     perror(str);
12     exit(-1);
13 }
14
15
16 int main(void)
17 {
18     int pfd[2],len;
19     pid_t pid;
20     char buf[4096];
21
22
23     if(pipe(pfd) < 0)
24         sys_err("pipe");
25
26
27     if((pid = fork()) < 0)
28         sys_err("fork");
29     else if(pid > 0)
30     {
31         //in paren
32         //父进程: 关闭读端, 进行写端
33         close(pfd[0]);
34         while(1)
35         {
36             len = read(STDIN_FILENO,buf,sizeof(buf));
37             if(len < 0)
38                 sys_err("read");
39             if(len == 0)
40                 break;
41             if(write(pfd[1],buf,len) < 0)
42                 sys_err("write");
43             if(read(pfd[0],buf,len) < 0)
44                 sys_err("read");
45             if(len == 0)
46                 break;
47         }
48     }
```



```

37         write(pfd[1],buf,len);
38     }
39     close(pfd[1]);
40
41
42 }
43 else
44 {
45     //in child
46     //子进程：关闭写端，进行读端
47     close(pfd[1]);
48     int flags;
49     int fd = open("abc",O_CREAT|O_RDWR,0766);
50     if(fd < 0)
51         sys_err("open");
52     //将读端设置为非阻塞
53     flags = fcntl(pfd[0],F_GETFL);
54     flags |= O_NONBLOCK;
55     fcntl(pfd[0],F_SETFL,flags);
56     while(1)
57     {
58         len = read(pfd[0],buf,sizeof(buf));
59         if(len < 0)
60         {
61             printf("no word\n");
62             sleep(1);
63             continue;
64         }
65         write(fd,buf,len);
66     }
67     close(pfd[1]);
68     close(fd);
69 }
70 return 0;
71 }

```

5、有名管道 (fifo)

创建一个有名管道，解决无血缘关系的进程通信， fifo:

```
1 #include <sys/types.h>
```

```

2 #include <sys/stat.h>
3
4
5 int mkfifo(const char *pathname, mode_t mode); //成功返回 0, 失败返回-1, 并设置errno

```

mode & ~umask

mkfifo既有命令也有函数。

有名管道 既可以标记内核的管道，打开有名管道，相当于打开了内核的管道。

当只写打开FIFO管道时，如果没有FIFO，没有读端打开，则open写打开会阻塞。（写阻塞会在open之前阻塞，只有读的时候才打开管道）

FIFO内核实现时支持双向通信。（pipe单向通信，因为父子进程共享同一个file结构体）

FIFO可以一个读端，多个写端；也可以一个写端，多个读端。

1、mkfifo_w.c (写)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <fcntl.h>
7
8
9 void sys_err(char *str)
10 {
11     perror(str);
12     exit(1);
13 }
14
15
16 int main(int argc, char *argv[])
17 {
18     int fd;
19     int n = 5;
20     char buf[1024];
21     if(argc < 2)
22     {

```

```

23     printf("./a.out filename\n");
24     exit(1);
25 }
26 if(access(argv[1],F_OK))
27     if(mkfifo(argv[1],0777) < 0)
28         sys_err("mkfifo");
29
30
31 printf("begin open\n");
32 if((fd = open(argv[1],O_WRONLY)) < 0)
33     sys_err("open");
34 printf("after open\n");
35
36
37 while(n--)
38 {
39     sprintf(buf,"yzmin %d\n",n);
40     write(fd,buf,strlen(buf));
41     sleep(1);
42 }
43 close(fd);
44
45
46 return 0;
47 }

```

2、mkfifo_r.c (读)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/stat.h>
6 #include <sys/types.h>
7 #include <fcntl.h>
8
9
10 void sys_err(char *str)
11 {
12     perror(str);
13     exit(1);

```

```
14 }
15
16
17 int main(int argc, char *argv[])
18 {
19     int fd, len;
20     int n = 5;
21     char buf[1024];
22     if(argc < 2)
23     {
24         printf("./a.out filename\n");
25         exit(1);
26     }
27
28
29     if(access(argv[1], F_OK))
30         if(mkfifo(argv[1], 0777) < 0)
31             sys_err("mkfifo");
32
33
34     if((fd = open(argv[1], O_RDONLY)) < 0)
35         sys_err("open");
36
37
38     while(1)
39     {
40         len = read(fd, buf, sizeof(buf));
41         if(len < 0)
42         {
43             sys_err("read");
44             break;
45         }
46         write(STDOUT_FILENO, buf, len);
47
48
49     }
50     close(fd);
51
52
53     return 0;
54 }
```

6、内存共享映射

1、mmap/munmap

mmap可以把磁盘文件的一部分直接映射到内存，这样文件中的位置直接就有对应的内存地址，对文件的读写可以直接用指针来做，而不需要read/write函数。

0地址是不允许操作的。第一个内存页 4 K，不允许读写操作。

```
1 #include <sys/mman.h>
2
3
4 void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
5 int munmap(void *addr, size_t length);
```

1、参数解析：

1、addr 表示指定内存映射的起始地址，通常置NULL，代表系统自动选定地址，映射成功则返回地址。

2、length 申请内存的长度，字节为单位。

3、prot 申请内存的读写权限：

- 1、PROT_EXEC 可执行
- 2、PROT_READ 可读取
- 3、PROT_WRITE 可写入
- 4、PROT_NONE 不可访问

4、flags

1、共享映射（MAP_SHARED）：同步到磁盘，多个进程中共享映射内存内容

2、私有映射（MAP_PRIVATE）：磁盘内容拷贝到映射内存，多进程中不共享映射内存内容，不同步到磁盘中

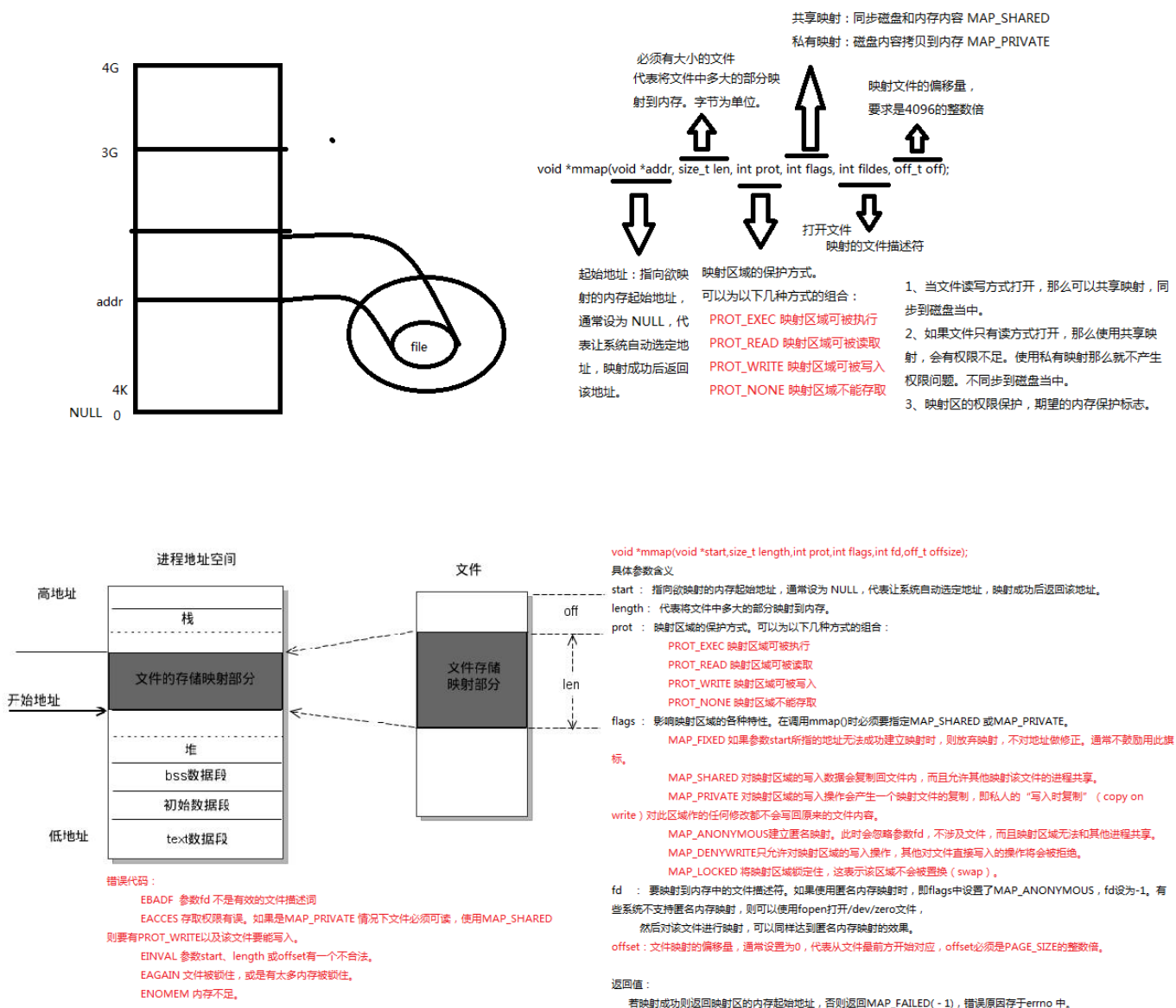
5、fd 打开的文件描述符

6、offset 映射文件内容的偏移量，要求是4096的整数倍

2 如果 mmap 成功则返回映射基地址（地址基址和找之间的区域） 如果出错则返回错误码

2、如果mmap成功则返回映射目标地址（地址是堆和栈之间的区域），如果出错则返回常量MAP_FAILED。当进程终止时，该进程的映射内存会自动解除，也可以调用munmap解除映射。munmap成功返回0，出错返回-1。

3、文件只有读权限，共享映射时，有读写权限，这样的映射则失败。



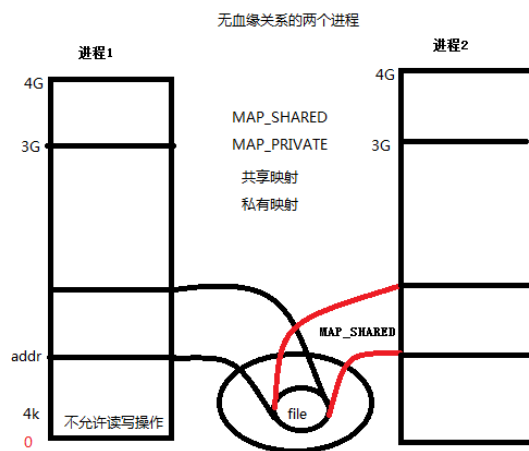
2、mmap.c

```
1 #include <stdio.h>
2 #include <sys/stat.h>
3 #include <sys/types.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <fcntl.h>
7 #include <sys/mman.h>
```

```
8
9
10 void sys_err(char *str)
11 {
12     perror(str);
13     exit(-1);
14 }
15
16
17 int main(void)
18 {
19     int fd,len;
20     char *mm;
21     if ((fd = open("abc",O_RDWR)) < 0)
22     {
23         sys_err("open");
24     }
25
26
27     len = lseek(fd,0,SEEK_END);
28     //mm = (char*)mmap(NULL,len,PROT_READ|PROT_WRITE,MAP_PRIVATE,fd,0);
29     mm = (char*)mmap(NULL,len,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0);
30     if(mm == MAP_FAILED)
31     {
32         sys_err("mmap");
33     }
34     printf("%s\n",mm);
35     char *bak = mm;
36     while(*mm!='\0')
37     {
38         *mm = toupper(*mm);
39         mm++;
40     }
41
42
43     if(munmap(bak,len)<0)//释放内存必须是内存首地址
44     {
45         sys_err("munmap");
46     }
47     close(fd);
48     return 0;
49 }
```

7、进程通信——内存映射

1、无血缘关系的进程



1、mmap_shared_w.c (写)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/mman.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7
8
9 void sys_err(char *str)
10 {
11     perror(str);
12     exit(1);
13 }
14
15
16 int main(void)
17 {
18     int fd, len, i = 0;
19     char *mm;
20
21
22     if((fd = open("abc", O_RDWR | O_CREAT, 0777)) < 0)
```



```

23     sys_err("open");
24     ftruncate(fd,4096);
25
26
27     mm = (char*)mmap(NULL,4096,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0);
28     if(mm == MAP_FAILED)
29         sys_err("mmap");
30
31
32     while(i < 4096)
33     {
34         *(mm+i) = 'a' + i;
35         i++;
36         sleep(1);
37     }
38
39
40     if(munmap(mm,4096) < 0)
41     {
42         sys_err("munmap");
43     }
44     close(fd);
45
46
47     return 0;
48 }

```

2、mmap_shared_r.c (读)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/mman.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7
8
9 void sys_err(char *str)
10 {
11     perror(str);
12     exit(1);

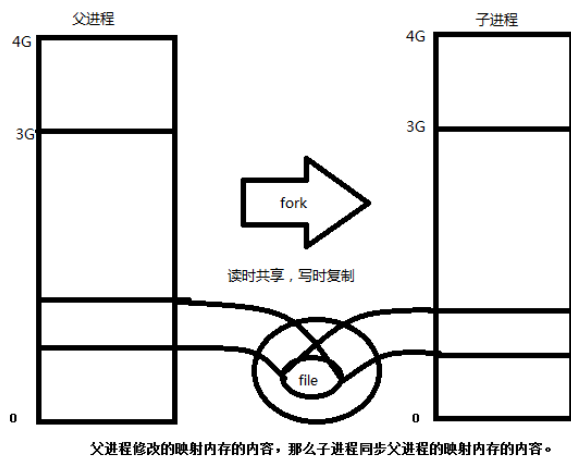
```

```

13 }
14
15
16 int main(void)
17 {
18     int fd,len,i = 0;
19     char *mm;
20
21
22     if((fd = open("abc",O_RDWR|O_CREAT,0777))< 0)
23         sys_err("open");
24
25
26     mm = (char*)mmap(NULL,4096,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0);
27     if(mm == MAP_FAILED)
28         sys_err("mmap");
29
30
31     while(1)
32     {
33         putchar(*mm+i);
34         fflush(stdout);
35         i++;
36         sleep(1);
37     }
38
39
40     if(munmap(mm,4096) < 0)
41     {
42         sys_err("munmap");
43     }
44     close(fd);
45
46
47     return 0;
48 }

```

2、有血缘关系的进程



1、mmap_fork1.c (普通)

```

1 #include <stdio.h>
2 #include <sys/mman.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <fcntl.h>
7 #include <stdlib.h>
8
9
10 int main(void)
11 {
12     pid_t pid;
13     char *mm,buf[4096];
14
15
16     int fd,i = 0;
17     fd = open("tmpfile",O_CREAT|O_RDWR,0777); //创建文件
18
19
20     ftruncate(fd,4096); //拓展文件
21     mm = mmap(NULL,4096,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0); //内存映射
22     close(fd);
23     unlink("tmpfile"); //删除，等到进程结束才删除
24
25
26     pid = fork(); //多进程
27     if(pid > 0)
28     {
29         while(1)

```

```

29     while(1)
30     {
31         sprintf(mm,"yzmin %d\n",i++);
32         sleep(1);
33     }
34     munmap(mm,4096); //分别关闭
35 }
36 else if(pid == 0)
37 {
38     while(1)
39     {
40         printf("%s\n",mm);
41         sleep(1);
42     }
43     munmap(mm,4096);
44 }
45 else
46 {
47     perror("fork");
48     exit(1);
49 }
50 return 0;
51 }

```

2、mmap_fork2.c(结构体)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7 #include <sys/mman.h>
8 #define PARENT_HAND 1
9 #define CHILD_HAND 2
10
11
12 struct FLAG
13 {
14     int flag;
15     char buf[2048];
16 };

```

```

16 },
17
18
19 int main(void)
20 {
21     pid_t pid;
22     struct FLAG *mm;
23     int fd,i = 0;
24
25
26     fd = open("tmpfile",O_CREAT|O_RDWR,0777);
27     ftruncate(fd,4096);
28     //父子各一份，同时指向同一磁盘空间
29     mm = (struct FLAG*)mmap(NULL,4096,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0); //MAP_PRI
30     mm->flag = PARENT_HAND;
31     unlink("tmpfile");
32     pid = fork();
33     if(pid > 0)
34     {
35         while(1)
36         {
37             if(mm->flag == PARENT_HAND)
38             {
39                 sprintf(mm->buf,"yzmin%d\n",i++);
40                 mm->flag = CHILD_HAND;
41             }
42             sleep(1);
43         }
44         munmap(mm,4096);
45     }
46     else if(pid == 0)
47     {
48         while(1)
49         {
50             if(mm->flag == CHILD_HAND)
51             {
52                 printf("%s\n",mm->buf);
53                 mm->flag = PARENT_HAND;
54             }
55             sleep(1);
56         }
57         munmap(mm,4096);
58     }
59     else

```

```

60 {
61     perror("fork");
62     exit(-1);
63 }
64 return 0;
65 }

```

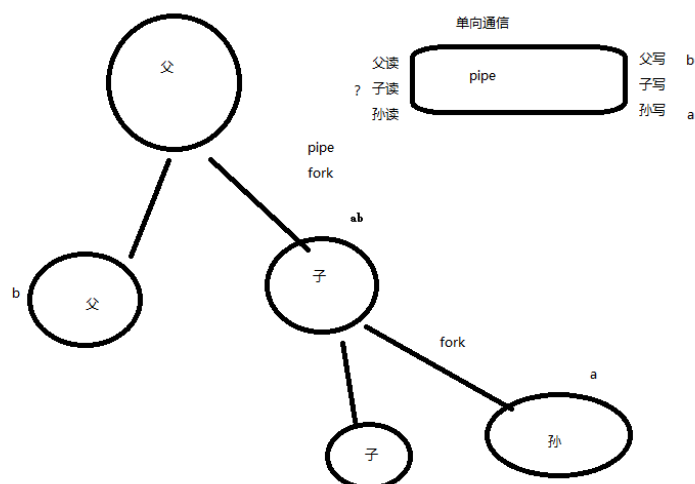
3、总结：

- 1、用于进程间通信时，一般设计成结构体，来传输通信的数据
- 2、进程间通信的文件，应该设计成临时文件
- 3、当报总线错误时，优先查看共享文件是否有存储空间

4、思考：

- 1. 如果mem++，munmap可否成功？ 不成功
- 2. 如果open时O_RDONLY, mmap时PROT参数指定PROT_READ|PROT_WRITE会怎样？ 没有权限
- 3. 如果文件偏移量为1000会怎样？ 映射失败
- 4. 如果不检测mmap的返回值，会怎样？ 如果出错，后面代码无法执行
- 5. mmap什么情况下会调用失败？ 权限不够，参数不对
- 6. 对mem越界操作会怎样？
- 7. 文件描述符先关闭，对mmap映射有没有影响？ 没有

8、多进程的管道通信



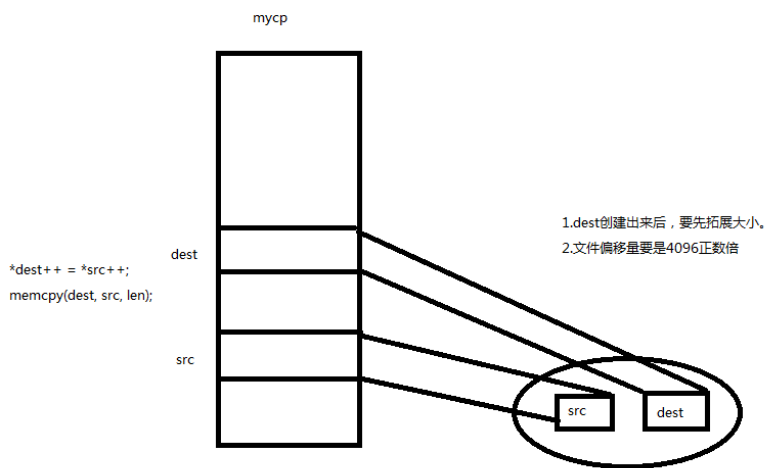
```
1  /*爷孙进程通信*/
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5  #include <fcntl.h>
6  #include <string.h>
7  #include <sys/wait.h>
8  #include <sys/types.h>
9
10
11 void sys_err(char *str)
12 {
13     perror(str);
14     exit(1);
15 }
16
17
18 int main(int argc, char *argv[])
19 {
20     pid_t pid;
21     int pfd[2];
22     char buf[1024];
23     int len;
24
25
26     if(pipe(pfd) < 0)
27         sys_err("pipe");
28
29
30     pid = fork();
31     if(pid > 0)
32     {
33         //爷写子读
34         close(pfd[0]);
35         sprintf(buf, "yzmin parent ppid %d, pid %d\n", getppid(), getpid());
36         write(pfd[1], buf, strlen(buf));
37         close(pfd[1]);
38
39
40
```

```

40
41
42     }
43     else if(pid == 0)
44     {
45         pid = fork();
46         if(pid == 0)
47         {
48             //父写子写，自己读
49             close(pfd[0]);
50             sprintf(buf, "sun write ppid %d, pid %d\n", getppid(), getpid());
51             write(pfd[1], buf, strlen(buf));
52             close(pfd[1]);
53             exit(0);
54         }
55         else if(pid > 0)
56         {
57             //孙写父读
58             close(pfd[1]);
59             sleep(3);
60             len = read(pfd[0], buf, sizeof(buf));
61             write(STDOUT_FILENO, buf, len);
62             exit(1);
63         }
64         else
65         {
66             sys_err("fork");
67         }
68     }
69     else
70     {
71         sys_err("fork");
72     }
73     wait(NULL);
74
75
76     return 0;
77 }

```

9、mmap内存共享——文件复制



```

1 //mmap_cp.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 #include <sys/stat.h>
7 #include <sys/mman.h>
8 #include <fcntl.h>
9 #include <string.h>
10
11
12 #define LEN 4096
13
14
15 void sys_err(char *str)
16 {
17     perror(str);
18     exit(1);
19 }
20
21
22 int main(int argc, char *argv[])
23 {
24     int srcfd, destfd;
25     int size;
26     char *mm;
27     if(argc < 3)
28     {
29         printf("./a.out src dest\n");
30         exit(-1);

```

```

31     }
32     if(!access(argv[1],F_OK))
33     {
34         srcfd = open(argv[1],O_RDONLY);
35         size = lseek(srcfd,0,SEEK_END);
36     }
37     else
38     {
39         printf("srcfile bu cunzai\n");
40         exit(1);
41     }
42     if(srcfd < 0)
43         sys_err("open src");
44
45
46     if(!access(argv[2],F_OK))
47         destfd = open(argv[2],O_RDWR|O_TRUNC);
48     else
49         destfd = open(argv[2],O_CREAT|O_WRONLY|O_TRUNC,0664);
50
51
52     if(destfd < 0)
53         sys_err("open destfd");
54
55
56     ftruncate(destfd,size);
57
58
59     int count = size / LEN;
60     if(size % LEN == 0)
61     {
62         int i;
63         for(i = 0;i < count; i++)
64         {
65             mm = mmap(NULL,LEN,PROT_READ|PROT_WRITE,MAP_PRIVATE,srcfd,LEN*i);
66             lseek(destfd,i*LEN,SEEK_SET);
67             write(destfd,mm,LEN);
68             munmap(mm,LEN);
69         }
70     }
71     else
72     {
73         int last_len = size % LEN;
74         int i;

```

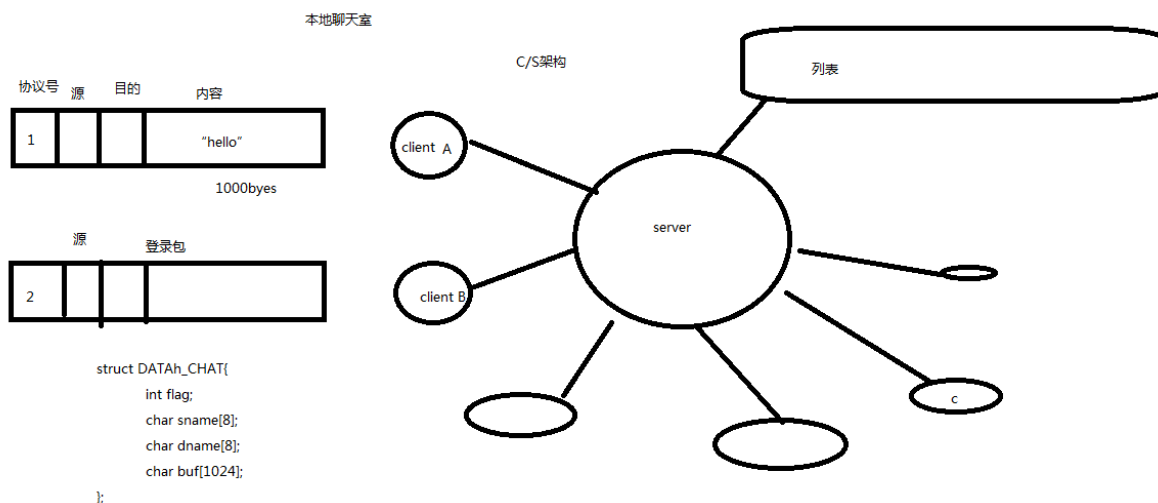
```

74     int i;
75
76
77     for(i = 0; i < count; i++)
78     {
79         mm = mmap(NULL, LEN, PROT_READ|PROT_WRITE, MAP_PRIVATE, srcfd, LEN*i);
80         lseek(destfd, i*LEN, SEEK_SET);
81         write(destfd, mm, LEN);
82         munmap(mm, LEN);
83     }
84     mm = mmap(NULL, last_len, PROT_READ|PROT_WRITE, MAP_PRIVATE, srcfd, LEN*count);
85     lseek(destfd, count*LEN, SEEK_SET);
86     write(destfd, mm, last_len);
87     munmap(mm, last_len);
88
89
90 }
91 close(srcfd);
92 close(destfd);
93
94
95 return 0;
96 }

```

10、本地聊天室原理

1、网络聊天室



2、本地聊天室

